

GETTING STARTED WITH C++

Joel FALCOU



A little history ...

At the Beginning:

- **1978**: Bjarne Stroustrup works on Simula67, which turns out to be not very efficient.
- **1980**: The C with Classes project, object-oriented C compiled via CFront.
- **1983**: C with Classes becomes C++.
- **1985**: Publication of The C++ Programming Language.
- **1998**: First standardization of C++: ISO/IEC 14882:1998.

The Vengeance :

- **2005** : Release of C++ Technical Report 1, which will become C++0x.
- **2011** : Ratification of C++0x as C++11.
- **2014** : Ratification of C++14.
- **2017** : Ratification of C++17.
- **2020,23,26** : Next Milestones (well underway)

Why C++ ?

'Make simple things simple'

- Simple tasks should be expressed simply.
- Complex tasks need not be insurmountable.

C++ : An Extensible Language

- Combinations of language elements must behave intuitively.
- Code abstraction level can be arbitrarily high.
- Reuse of components is encouraged.

C++ : Close to the Metal

- *'Don't pay for what you don't use'.*
- *'Zero Cost Abstraction'.*

The C++ Programming Model

A multi-paradigm language

- Imperative Programming
- Object Oriented
- Functional Programming

Modèle de programmation C++

A multi-paradigm language

- **Imperative Programming**
 - Code is a simple list of instructions.
 - Its fundamental elements are functions and data structures.
- **Object Oriented**
- **Functional Programming**

A multi-paradigm language

- **Imperative Programming**
- **Object Oriented**
 - The code is built around the definition of objects modeling the elements of the problem.
 - Its fundamental elements are classes and the relationships between these classes.
- **Functional Programming**

A multi-paradigm language

- **Imperative Programming**
- **Object Oriented**
- **Functional Programming**
 - The code is based on the use of functions and values.
 - There is no notion of memory or control structure.
 - Its fundamental elements are functions.

Modèle de programmation C++

A multi-paradigm language

- Imperative Programming
- Object Oriented
- Functional Programming

What to choose?

- All these styles can be used to solve everyday development problems.
- Some problems have simpler solutions in some of these styles.
- All contribute to making C++ an efficient language.

From code to executable

Definition of “source code”

- Source code is a set of text files.
- These file(s) are **readable by a human**.
- It best expresses the developer's intention.

Code/language relationship

- A **programming language** (such as C++) allows source code to be written using source code via formalized logical constructs.
- The source code is processed by a **compiler**, which analyzes and verifies it and transforms it into elements that can be executed by the processor

Source Code - Example

my_program.cpp

- A program requires an **entry point**.
- This entry point is executed when the program is started.
- In C++, this entry point is called `main`.

```
1  main
```

Source Code - Example

my_program.cpp

- `main` is a **function**, a fundamental operational unit in C++.
- A function receives **arguments** as input.
- A function processes its arguments and, potentially, returns a **result**.
- `main`, in its simple form, receives no arguments and returns an integer (**int**)

```
1  int main()
```

Source Code - Example

my_program.cpp

- A function processes its arguments by executing the code it contains.
- A function's code is contained in its **body**, delimited by { }.
- `main` returns an integer equal to 0 via the **keyword** `return`.

```
1  int main()  
2  {  
3      return 0;  
4  }
```

Source Code - Example

my_program.cpp

- C++ provides pre-defined functions: **the standard library**
- These functions are declared in **header files**.
- Headers can be used via the `#include` directive
- For example, let's use the display functions provided by the `iostream` header.

```
1  #include <iostream>
2
3  int main()
4  {
5      std::cout << "Exécution réussie !" << std::endl;
6      return 0;
7  }
```

The Compilation Process

General principles

- Analyses and checks source code for correctness.
- Translates code into assembly language.
- Transforms assembler into machine code.
- Aggregates all necessary elements to produce an executable binary file.

The three stages of compilation

- Pre-processing.
- Code generation.
- Link editing.

The Compilation Process

General principles

- Analyzes and checks source code for correctness
- Translates code into “processor” language (assembler)
- Aggregates all necessary elements to produce an executable binary file

The three stages of compilation

- Pre-processing
 - Processes source text to make it understandable to the compiler
- Code generation
- Link editing

The Compilation Process

General principles

- Analyzes and checks source code for correctness
- Translates code into “processor” language (assembler)
- Aggregates all necessary elements to produce an executable binary file

The three stages of compilation

- Pre-processing
- Code generation
 - Analyzes code and generates binary code
- Link editing

The Compilation Process

General principles

- Analyzes and checks source code for correctness
- Translates code into “processor” language (assembler)
- Aggregates all necessary elements to produce an executable binary file

The three stages of compilation

- Pre-processing
- Code generation
- Link editing
 - Aggregates binary code fragments into an executable

The Compilation Process

Invoking the compiler

`g++ <source> -o <executable> <option>`

- **<source>** : list of source files to compile
- **<executable>** : name of executable program to be generated
- **<option>** : list of compiler options: optimization, language version, advanced error checking, architecture management, etc...

Examples

- Optimized compilation with in-depth checks
`g++ mon_programme.cpp -o mon_programme.prog -Wall -O3 -std=c++17`
- Compiling for debugging
`g++ mon_programme.cpp -o mon_programme.prog -g -std=c++17`

The Compilation Process

Invoking the compiler

```
g++ <source> -o <executable> <option>
```

- **<source>** : list of source files to compile
- **<executable>** : name of executable program to be generated
- **<option>** : list of compiler options: optimization, language version, advanced error checking, architecture management, etc...

Examples

- Some components require external code to function
- The -l option allows the link editor to access this pre-compiled code.

```
g++ mon_programme.cpp -o mon_programme.prog -Wall -O3 -std=c++17 -lpthread
```

Types and Variables

Notion of type

- Native types
- Advanced types
- Qualifiers
- Type inference

Notion of data structures

- Enumerated types
- Aggregated types

What you need to remember

- Which type for which task?
- Usefulness of qualifiers

Notion of Types

Native types

Definition

- These are **data types** managed directly by the language and processor.
- They represent the fundamental values that can be manipulated by a C++ program.

Contents

- Signed and unsigned integers
- Fixed-size integers
- Flexible-size integers
- Boolean values
- Floating-point numbers

Native types - Signed integers

Properties

- Representats of relative integers ($\mathbb{Z}$)
- **Their numerical amplitudes vary.**
- Support classic arithmetic operations
- Bit manipulation of these types should be avoided.

```
1 // Signed integers
2 signed char    c; // values between -128 and 127
3 signed short   s; // values between -32768 and 32767
4 signed int     i; // values between -2147483648 and 2147483647
5 signed long long l; // values between -9223372036854775808 and 9223372036854775807
```

Native types - Signed integers

Properties

- Representation of relative integers ($\mathbb{Z}$)
- Numerical amplitudes vary
- **Support classic arithmetic operations**
- Bit manipulation of these types should be avoided.

```
1  signed int    i = 4'000'000;
2  signed short  u = 40'000;
3  signed char   c = 60;
4
5  // Arithmetics
6  i = i + 3 * u;  // i = 4'120'000
7  u = -c;        // u = -60
8  c = 2*c;       // c = 120
9
10 // Range reduction
11 c = 2*c;       // c = -16
```

Native types - Unsigned integers

Properties

- Representation of a set of bits
- **Their numerical amplitudes vary**
- Support operations on bits
- Arithmetic operations on these types should be avoided.

```
1 // Unsigned Integers
2 unsigned char    c; // values between 0 and 255
3 unsigned short   s; // values between 0 and 65535
4 unsigned int     i; // values between 0 and 4294967295
5 unsigned long long l; // values between 0 and 18446744073709551615
```

Native types - Unsigned integers

Properties

- Representation of a set of bits
- Their numerical amplitudes vary
- **Support operations on bits**
- Arithmetic operations on these types should be avoided.

```
1  unsigned char some  = 0x3C;      // 60 in hexadecimal
2  unsigned char other = 0b01010101; // 85 in binary
3
4  unsigned char r = some & other;   // r = 0b00010100 = 20
5  unsigned char s = some | other;   // s = 0b01111101 = 125
6
7  unsigned char d = r >> 2;         // d = 5
8  unsigned short l = (s << 8) | r;  // l = 0b0111110100010100 = 32020
```

Native types - Fixed-size integers

Problem

- The size of native integers depends on the system
- How to ensure a minimum size?
- Use cases: saving to disk, sending over the network, etc...

Solution

- C++ provides a set of integer types of guaranteed size and sign
- Allows values to be shared between systems and architectures
- **Do not replace native types for arithmetic.**

Native types - Fixed-size integers

Usage

- Available in `#include <cstdint>`
- General form: `{u}int{8,16,32,64}_t`

```
1  #include <cstdint>
2
3  // 64-bits signed integer
4  std::int64_t s = 123456789098LL;
5
6  // 8-bits unsigned integer
7  std::uint8_t mask = 0b11110000;
```

Native types - Flexible integer sizes

Problem

- The value of certain quantities depends on the system
 - size of an object in memory
 - distance between two values in memory
- How to store these values in a variable of the right type?

Solution

- C++ provides a set of types whose size and sign vary automatically.
- **To be used systematically to store these values.**

Native types - Flexible integer sizes

Usage

- Available in `#include <cstdint>`.

```
1  #include <cstdint>
2
3  long x,y;
4
5  // std::size_t represents the size of an object in memory
6  std::size_t how_big = sizeof(x); // sizeof(x) returns the size of x in bytes
7
8  // std::ptrdiff_t represents the distance between two values in memory
9  std::ptrdiff_t how_far = &y - &x; // & indicates the position of a variable in memory
```


Native types - Boolean values

Objectives

- Introduce the `bool` type and the `true` and `false` values.
- Represent a “yes” or “no” value
- **To be used systematically** to represent a logical value

```
1  bool prop1 = true;    // logically true proposition
2  bool prop2 = false;   // logically false proposition
3
4  int i = 9;
5  bool is_big = i > 15; // comparison operations produce bool
6
7  // Logical combination operations
8  // ! : logical NOT
9  // && : logical AND
10 // || : logical OR
11 bool truth = (!is_big && prop1) || prop2;
```

Native types - Floating-point numbers

Problem

- Compromise between accuracy and performance
- Use of the IEEE754 standard
- “Hole” representation of ($\mathbb{R}$)

```
1 // 32-bits floating-point values - 7 significant digits
2 float x = 0.2541369f;
3
4 // 64-bits floating-point values - 16 significant digits
5 double x = 0.2541369780123548;
6
7 // Interaction with integers
8 float quotient_incorrect = 1 / 5; // = 0 integer division
9 float quotient_correct = 1 / 5.f; // = 0.2f real division
```

Advanced types

Definition

- **More complex** data types
- Allows you to manipulate higher-level abstractions
- Provided by the standard library

Contents

- Character string
- Fixed array size
- Dynamic array size
- Pair
- Tuple
- Set
- Dictionary

Definition

- C++ provides a type representing a character string.
- This string can only contain extended ASCII characters.
- Defined in the `<string>` header.

Interface

- Read/write access to elements
- Copy, Concatenate, Resize
- Search, extraction of substrings
- Automatic memory management

Advanced types - String

Default construction

```
1  #include <string>
2  #include <iostream>
3
4  int main()
5  {
6      // Default construction
7      std::string s;
8
9      // Access to size
10     std::cout << s.size() << "\n";
11 }
```

0

Advanced types - String

Advanced constructions

```
1  #include <string>
2  #include <iostream>
3
4  int main()
5  {
6      std::string sc = "this is text";
7      std::string sp("this is text",4);
8      std::string si( &sc[5], &sc[12]);
9      std::string sr(10, '*');
10
11     std::cout << sc << "\n" << sp << "\n";
12     std::cout << si << "\n" << sr << "\n";
13 }
```

```
this is text
this
is text
*****
```

Advanced types - String

Character handling

```
1  #include <string>
2  #include <iostream>
3
4  int main()
5  {
6      std::string sc = "ceci est du texte";
7      std::cout << sc[3] << "\n";          // Read
8
9      sc[0] = 'C';                          // Write
10     std::cout << sc << "\n";
11
12     sc.push_back('!');                     // Add at the end
13     std::cout << sc << "\n";
14 }
```

```
i
Ceci est du texte
Ceci est du texte!
```

Advanced types - String

Chain handling

```
1  #include <string>
2  #include <iostream>
3
4  int main()
5  {
6      std::string st = "top";
7      std::string sb = "bingo!";
8
9      auto stb = st + " " + sb;           // Concatenation
10     std::cout << stb << "\n";
11
12     stb.clear();                         // Clearing
13     std::cout << "'" << stb << "'" << "\n";
14 }
```

```
top bingo!
''
```


Advanced types - String

Subchain search/extraction

```
1  #include <string>
2  #include <iostream>
3
4  int main()
5  {
6      std::string sc = "ceci est du texte";
7
8      auto sub = sc.substr(9,2);
9      std::cout << sub << "\n";
10
11     std::size_t n = sc.find("est");
12     if (n == std::string::npos) std::cout << "pas trouvé\n";
13     else
14         std::cout << "trouvé: " << sc.substr(n) << '\n';
15 }
```

```
du
trouvé: est du texte
```

Advanced types - Fixed size table

Definition

- C++ provides a type representing a succession of fixed-size values.
- These elements are stored contiguously in memory
- This type represents a **array**.
- Defined in the `<array>` header

```
//    Type of table values
//      |
//      v
std::array< int, 42 > p;
//      ^
//      |
//      Number of elements
```

Advanced types - Fixed size table

Usage

```
1  #include <array>
2
3  int main()
4  {
5      std::array<double, 3> origin = { 0,0,0 };
6      std::array<double, 3> p1 = {1.5,-0.6,2.3 };
7      std::array<double, 3> p2 = p1;
8
9      std::cout << origin.size() << "\n";
10     std::cout << p2[0] << " " << p2[1] << " " << p2[2] << "\n";
11 }
```

```
3
1.5 -0.6 2.3
```

Advanced types - Dynamic size table

Definition

- C++ provides a type representing a succession of variable-size values.
- These elements are stored contiguously in memory
- This type represents a **table** whose “volume” can vary
- Defined in the `<vector>` header

```
//      Type of table values
//      |
//      v
std::vector<float> v;
```

Advanced types - Dynamic size table

Default construction

```
1  #include <vector>
2  #include <iostream>
3
4  int main()
5  {
6      // Default construction
7      std::vector<float> s;
8
9      // Access to size
10     std::cout << s.size() << "\n";
11 }
```

0

Advanced types - Dynamic size table

Advanced constructs

```
1  #include <vector>
2  #include <iostream>
3
4  int main()
5  {
6      // 6 elements initialized by default
7      std::vector<float> v(6);
8
9      std::cout << v[0] << " " << v[1] << " " << v[2] << " ";
10     std::cout << v[3] << " " << v[4] << " " << v[5] << "\n";
11 }
```

0 0 0 0 0 0

Advanced types - Dynamic size table

Advanced constructs

```
1  #include <vector>
2  #include <iostream>
3
4  int main()
5  {
6      // 6 specific values
7      std::vector<float> v = {1.2f, 3.4f, 5.6f, 7.8f, 9.10f, 11.12f};
8
9      std::cout << v[0] << " " << v[1] << " " << v[2] << " ";
10     std::cout << v[3] << " " << v[4] << " " << v[5] << "\n";
11 }
```

1.2 3.4 5.6 7.8 9.1 11.12

Advanced types - Dynamic size table

Advanced constructs

```
1  #include <vector>
2  #include <iostream>
3
4  int main()
5  {
6      // 6 repeat values
7      std::vector<float> v(6,-42.69);
8
9      std::cout << v[0] << " " << v[1] << " " << v[2] << " ";
10     std::cout << v[3] << " " << v[4] << " " << v[5] << "\n";
11 }
```

-42.69 -42.69 -42.69 -42.69 -42.69 -42.69

Advanced types - Dynamic size table

Advanced constructs

```
1  #include <vector>
2  #include <iostream>
3
4  int main()
5  {
6      // 3 values extracted from a related source
7      std::vector<float> u = {6,5,4,3,2,1};
8      std::vector<float> v = { &u[1], &u[4] };
9
10     std::cout << v[0] << " " << v[1] << " " << v[2] << "\n";
11 }
```

5 4 3

Advanced types - Dynamic size table

Value manipulation

```
1  #include <vector>
2  #include <iostream>
3
4  int main()
5  {
6      std::vector<float> v = {0,0.5,1.5,3,6,36};
7      std::cout << v[3] << "\n";      // Read
8
9      v[1] = 10*v[1];                  // Write
10     std::cout << v[1] << "\n";
11
12     v.push_back(99);                 // Add at the end
13     std::cout << v[6] << "\n";
14 }
```

3
5
99

Advanced types - Pair

Definition

- A pair is a data type containing two elements of arbitrary type.
- It represents an aggregate of two values that are logically paired but have no particular but have no particular semantics.
- Defined in the <utility> or <tuple> header.
- These values are simply named **first** and **second**.

```
//      1st component
//      |
//      v
std::pair< int, float > p;
//      ^
//      |
//      2nd component
```

Advanced types - Pair

Default pair

```
1  #include <utility>
2  #include <iostream>
3
4  int main()
5  {
6      // Default construction
7      std::pair<int,float> p0;
8
9      // Modification
10     p0.second = 0.5f;
11
12     // Access
13     std::cout << p0.first << " " << p0.second << "\n";
14 }
```

0 0.5

Advanced types - Pair

Pair initialization

```
1  #include <utility>
2  #include <iostream>
3
4  int main()
5  {
6      // Initialization
7      std::pair<int,float> p1(5,3.8f);
8
9      // Access
10     std::cout << p1.first << " " << p1.second << "\n";
11 }
```

5 3.8

Advanced types - Pair

Pair initialization by conversion

```
1  #include <utility>
2  #include <iostream>
3
4  int main()
5  {
6      // Initialization
7      std::pair<int,float>    p1(65,3.8f);
8
9      // Implicit Conversion
10     std::pair<char,double>  p2(p1);
11
12     std::cout << p2.first << " " << p2.second << "\n";
13 }
```

A 3.8

Advanced types - Tuple

Definition

- The tuple (or N-tuplet) is a data type that generalizes the pair to aggregates of N values of arbitrary types
- As with `std::pair`, a tuple has no particular semantics.
- defined in the `<tuple>` header
- These elements are accessible via indexing known at compile time

```
1 //      1st component 3rd component
2 //          |           |
3 //          v           v
4 std::tuple< int, float, char > t;
5 //          ^
6 //          |
7 //      2nd component
```

Advanced types - Tuple

Default tuple

```
1  #include <tuple>
2  #include <string>
3  #include <iostream>
4
5  int main()
6  {
7      // Default construction
8      std::tuple<int,float, std::string> p0;
9
10     // Modification
11     std::get<2>(p0) = "Hello !";
12
13     // Access
14     std::cout << std::get<0>(p0) << " " << std::get<1>(p0) << " " << std::get<2>(p0) << "\n";
15 }
```

0 0 Hello !

Advanced types - Tuple

Initialization

```
1  #include <tuple>
2  #include <string>
3  #include <iostream>
4
5  int main()
6  {
7      // Manual initialization
8      std::tuple<int,float, std::string> p = { 3, 3.1415f, "pi" };
9
10     // Initialization by make_tuple
11     auto e = std::make_tuple( 2, 2.71828f, "euler" );
12
13     std::cout << std::get<2>(p) << " = " << std::get<1>(p) << "\n";
14     std::cout << std::get<2>(e) << " = " << std::get<1>(e) << "\n";
15 }
```

```
pi = 3.1415
euler = 2.71828
```

Advanced types - Tuple

Variable linking

```
1  #include <tuple>
2  #include <iostream>
3
4  int main()
5  {
6      float      value;
7      std::string name;
8
9      auto v = std::tie(value,name);
10
11     std::get<0>(v) = 1.61803f;
12     std::get<1>(v) = "golden";
13
14     std::cout << name << " = " << value;
15 }
```

golden = 1.61803

Advanced types - Tuple

Tuple manipulation

```
1  #include <tuple>
2  #include <iostream>
3
4  int main()
5  {
6      auto t1 = std::make_tuple("Hell");
7      auto t2 = std::make_tuple('o', " World", '!');
8
9      auto t12 = std::tuple_cat(t1, t2);
10     std::cout << std::get<0>(t12) << std::get<1>(t12)
11               << std::get<2>(t12) << std::get<3>(t12) << "\n";
12 }
```

Hello World!

Advanced types - Set

Definition

- A set is a type representing a sorted list of unique values.
- It provides suitable insert and search operations
- Defined in the `<set>` header

```
//    Type of values  
//    |  
//    v  
std::set< int > s;
```

Construction and insertion

```
1  #include <set>
2  #include <iostream>
3
4  int main()
5  {
6      std::set<std::string> animals;
7      animals.insert("horse");
8      animals.insert("dog");
9      animals.insert("zebra");
10     animals.insert("cat");
11
12     for(auto& str: animals) std::cout << str << "\n";
13 }
```

cat
dog
horse
zebra

Advanced types - Set

Construction and search

```
1  #include <set>
2  #include <iostream>
3
4  int main()
5  {
6      std::set<std::string> numbers{"un", "deux", "trois", "sept", "onze"};
7
8      auto n = numbers.find("trois");
9      if (n != numbers.end()) std::cout << "Trouvé " << (*n) << '\n';
10     else std::cout << "Pas trouvé :(\n";
11 }
```

Trouvé trois

Advanced types - Dictionary

Definition

- A dictionary is a type representing a list of associations between **keys** and **values**.
- It provides suitable insertion and search operations.
- Defined in the `<map>` header

```
//      Key type
//      |
//      v
std::map< int, std::string > m;
//      ^
//      |
//      Type of values
```

Advanced types - Dictionary

Constructions

```
1  #include <map>
2  #include <string>
3
4  int main()
5  {
6      std::map<std::string, int> m; // Default construction
7
8      m["pierre"] = 27;           // Insert pair {key,value}
9      m["paul"]   = 19;           // m["key"] = value
10
11     std::map<std::string, float> csts // Enumeration construction
12     {
13         // { clé , valeur}
14         {"pi"      , 3.14152f },
15         {"euler"   , 2.71828f },
16         {"golden"  , 1.61803f }
17     };
18 }
```


Advanced types - Dictionary

Search

```
1  #include <map>
2  #include <string>
3  #include <iostream>
4
5  int main()
6  {
7      std::map<std::string, float> csts{ {"pi", 3.14152f }, {"euler", 2.71828f } };
8
9      // Direct access
10     std::cout << csts["pi"] << "\n";
11
12     // /\ Incorrect if key does not exist
13     std::cout << csts["golden"] << "\n";
14 }
```

3.14152

0

Advanced types - Dictionary

Search

```
1  #include <map>
2  #include <string>
3  #include <iostream>
4
5  int main()
6  {
7      std::map<std::string, float> csts{ {"pi", 3.14152f }, {"euler", 2.71828f } };
8
9      // find behaves consistently in the event of a missing key
10     auto v = csts.find("pi");
11
12     // The result gives access to the key and the value
13     if (v != csts.end()) std::cout << "Trouvé: " << v->first << " " << v->second << '\n';
14     else                 std::cout << "Pas trouvé\n";
15 }
```

Trouvé: pi 3.14152

Type manipulation

Type qualifiers

Objectives

- Reinforce type semantics
- Add a property to a type
- Clarify the variable's relationship with the environment

Contents

- Immutability
- Reference
- Pointer

Type qualifiers - Immutability

Definition

- An **immutable** variable cannot change its value.
- It must be defined before it can be used.
- Immutability is expressed by the keyword **const**.

```
1  int      x = 10; // variable
2  int const y = 42; // constant variable
3
4  // This code is valid
5  x = 9;
6  x = 2*y;
7
8  // This code does not compile :
9  // > error: assignment of read-only variable 'y'
10 y = 4;
```

Type qualifiers - Reference

Definition

- A reference is an **alias** for an existing variable.
- It must be **required** to be linked to another variable.
- Manipulating a reference directly manipulates the referenced variable.
- A reference can add immutability.

```
1  std::string s = "Ex";      // original variable
2  std::string const& r2 = s;  // r2 is an immutable alias of s
3
4  std::string& r1 = s;       // r1 is an alias of s
5  r1 += "emple";            // Modifying r1 modifies s
6
7  // This code does not compile :
8  std::string& r0;           // > error: 'r0' declared as reference but not initialized
9  r2 += "!";                // > error: cannot modify through reference to const
```

Type qualifiers - Pointer

Definition

- A pointer represents the position of a variable in memory via the & operator
- It provides indirect access to the associated value via **dereferencing**
- Unlike a reference, a pointer can refer to nothing.
- Unlike a reference, a pointer can change target
- The immutability of a pointer is independent of that of its target.

```
1  std::string s = "Ex";    // original variable
2  std::string* p1 = &s;    // r1 is a pointer to a string
3
4  *p1 += "emple";          // Modifying *r1 modifies s
```

Type qualifiers - Pointer

Definition

- A pointer represents the position of a variable in memory via the & operator
- It provides indirect access to the associated value via **dereferencing**
- **Unlike a reference, a pointer can refer to nothing.**
- **Unlike a reference, a pointer can change target**
- The immutability of a pointer is independent of that of its target.

```
1  std::string s = "Ex";    // original variable
2  std::string* p1 = &s;    // p1 is a pointer to a string
3
4  std::string* p0 = nullptr; // p0 targets "nothing"
5  p0 = p1;                // p0 now targets the same variable as p1
6
```


Qualificateurs de types - Pointeur

Définition

- A pointer represents the position of a variable in memory via the & operator
 - It provides indirect access to the associated value via **dereferencing**
 - Unlike a reference, a pointer can refer to nothing.
 - Unlike a reference, a pointer can change target
 - **The immutability of a pointer is independent of that of its target.**

```
1  int var = 5;           // original variable
2
3  int* pi = &var;        // pointer to an int
4  int const* pci = &var; // pointer to an constant int
5  int* const cpi = &var; // constant pointer to an int
6  int const* const cpci = &var; // constant pointer to an constant int
```

Type alias

Objectives:

- Some types have very long names, making it difficult to read the code.
- Some types are complex to express
- C++ offers a lightweight type renaming system

```
1  #include <array>
2
3  // using new_type = old_type
4  using mass      = float;
5  using point3D   = std::array<double, 3>;
6
7  // direct usage
8  point3D p1 = { 0,1.5,3. };
9  mass ppl_weight = 51.5f;
```

Type inference

Problems

- Declaring a variable requires expressing its type.
- In the case of initializations, the type is already known by the compiler.
- In other contexts, the type of a variable cannot be expressed by a human.

```
1  #include<vector>
2
3  float const f = 3.f; // 3.f already indicates it's a float
4
5  std::size_t sz = sizeof(f); // do you remember the return type of sizeof
6
7  std::vector<std::uint8_t> bytes(sz);
8
9  // This code fragment is too long to indicate that b is the beginning of bytes
10 std::vector<std::uint8_t>::iterator b = bytes.begin();
```

Type inference

Solution: the **auto** keyword

- **auto** replaces the type of a variable during initialization
- The variable's type is then inferred as the type of its initializer's value.
- **auto** can be used with qualifiers

```
1  #include<vector>
2
3  auto const f = 3.f;
4
5  auto sz = sizeof(f);
6
7  std::vector<std::uint8_t> bytes(sz);
8
9  auto b = bytes.begin();
```

Inputs/Outputs

Stream templates

- Console feed
- Chain feeds
- File streams

Stream handling

- Error handling
- Formatting for data structures
- File management

Things to remember

- Which flow for which task?
- Safe I/O management

Stream models

Principles

- Input/output homogenization
- Classic cases: screen, keyboard, file
- Less classic cases: strings
- Designed to be extensible

Interface :

- Inserted into a stream:

```
stream << value
```

- Extraction from a stream:

```
stream >> value
```


Principles

- Provide objects capable of handling basic input/output flows
- Notch, keyboard, error display
- Accessible via `#include <iostream>`

Stream provided

- `std::cin` : keyboard input
- `std::cout` : console output
- `std::cerr` : error console output

Implementation

```
1  #include <iostream>
2
3  int main()
4  {
5      std::cout << "Give a positive real:\n";
6
7      float x;
8      std::cin >> x;
9
10     if(x < 0.f)
11         std::cerr << "Error : " << x << " is negative\n";
12 }
```

Chain streams

Principles

- Adapting strings as streams
- A string contains extractable data
- A string can be constructed by inserting it into a stream
- Accessible via `#include <sstream>`

Provided streams

- `std::ostringstream` : insertion into a string
- `std::istringstream` : extraction into a string

Chain streams

Usage

```
1  #include <iostream>
2  #include <string>
3  #include <sstream>
4
5  int main()
6  {
7      std::ostringstream data;
8      data << "'test: " << 3.54f << ' ' << 15784 << "'\n";
9
10     std::string s = data.str();
11     std::cout << s;
12 }
```

```
'test: 3.54 15784'
```

Chain streams

Usage

```
1  #include <iostream>
2  #include <string>
3  #include <sstream>
4
5  int main()
6  {
7      int mn, mx;
8      double v;
9      std::string name;
10
11     std::istringstream in("9 -8 7.5 camera");
12     in >> mx >> mn >> v >> name;
13
14     std::cout << name << " " << v << " " << mn << " " << mx << "\n";
15 }
```

camera 7.5 -8 9

Chain streams

Usage

```
1  #include <iostream>
2  #include <sstream>
3  #include <string>
4  #include <map>
5
6  int main()
7  {
8      std::map<std::string, int> histogram;
9      std::istringstream ss("des papous papa ont des poux papa papous");
10
11     std::string w;
12     while(ss >> w)
13         histogram[w]++;
14
15     for(auto p : histogram)
16         std::cout << p.first << " -> " << p.second << "\n";
17 }
```

Control structures

Repetitive control flows

- Deterministic flow
- Conditional flow

Conditional control flows

- Conditional block
- Selection block

Things to remember

- Semantics of control structures
- Large-grain organization of a program

Repetitive control flows

Deterministic flows

Objective:

- Repeat a block of code a certain number of times
- The number of repetitions is known in advance.
- relies on the use of one or more countdown variables

General form:

```
1  for( initialization; stop condition; update)
2  {
3      // code to be repeated
4  }
```

Deterministic repetitions

Example: sum of 1 to N

```
1  #include <iostream>
2
3  int main()
4  {
5      int resultat = 0, N = 10;
6
7      for (int i = 1; i <= N; ++i)
8      {
9          resultat = resultat + i;
10     }
11
12     std::cout << "Somme(1,10) = " << resultat << "\n";
13 }
```

Somme(1,10) = 55

Deterministic repetitions

Example: multiplicative update

```
1  #include <iostream>
2
3  int main()
4  {
5      // MAJ multiplicative
6      for (int i = 1; i < 65536; i *= 16)
7      {
8          std::cout << i << "\n";
9      }
10 }
```

```
1
16
256
4096
```

Deterministic repetitions

Traversing an array structure

```
1  #include <iostream>
2  #include<array>
3
4  int main()
5  {
6      int resultat = 0;
7      std::array<int, 10> vs = { 1,2,3,4,5,6,7,8,9,10 };
8
9      for (int i = 0; i < N; i++)
10     {
11         resultat = resultat + vs[i];
12     }
13
14     std::cout << "Somme(1,10) = " << resultat << "\n";
15 }
```

Somme(1, 10) = 55

Deterministic repetitions

Traversing an array structure

```
1  #include <iostream>
2  #include <array>
3
4  int main()
5  {
6      int resultat = 0;
7      std::array<int, 10> vs = { 1,2,3,4,5,6,7,8,9,10 };
8
9      for (int v : vs)
10     {
11         resultat = resultat + v;
12     }
13
14     std::cout << "Somme(1,10) = " << resultat << "\n";
15 }
```

Somme(1, 10) = 55

Deterministic repetitions

Traversing an array structure

```
1  #include <iostream>
2  #include <array>
3
4  int main()
5  {
6      int resultat = 0;
7      std::array<int, 10> vs = { 1,2,3,4,5,6,7,8,9,10 };
8
9      for (auto v : vs)
10     {
11         resultat = resultat + v;
12     }
13
14     std::cout << "Somme(1,10) = " << resultat << "\n";
15 }
```

Somme(1, 10) = 55

Deterministic repetitions

Traversing an array structure

```
1  #include <iostream>
2  #include <array>
3
4  int main()
5  {
6      double resultat = 0;
7      std::array<double, 10> vs = { 1,2,3,4,5,6,7,8,9,10 };
8
9      for (auto v : vs)
10     {
11         resultat = resultat + v;
12     }
13
14     std::cout << "Somme(1,10) = " << resultat << "\n";
15 }
```

Somme(1, 10) = 55

Deterministic repetitions

Traversing an array structure

```
1  #include <iostream>
2  #include <vector>
3
4  int main()
5  {
6      double resultat = 0;
7      std::vector<double> vs{ 1,2,3,4,5,6,7,8,9,10 };
8
9      for (auto v : vs)
10     {
11         resultat = resultat + v;
12     }
13
14     std::cout << "Somme(1,10) = " << resultat << "\n";
15 }
```

Somme(1, 10) = 55

Conditional repetitions

Objective:

- Repeat a block of code depending on an external condition.
- The number of repetitions is not necessarily known in advance.
- Relies on the use of a manually updated condition

General forms - 0+ iterations:

```
1  while( condition )  
2  {  
3      // code to be repeated as long as condition evaluates to `true`.  
4  }
```

Conditional repetitions

Objective:

- Repeat a block of code depending on an external condition.
- The number of repetitions is not necessarily known in advance.
- Relies on the use of a manually updated condition

General forms - 1+ iterations:

```
1  do
2  {
3      // code to be repeated as long as condition evaluates to `true`
4  } while( condition );
```

Conditional repetitions

Example - Value search

```
1  #include <iostream>
2  #include <vector>
3
4  int main()
5  {
6      std::vector<float> x = { 3, 4, 5, 8, 7, 99, 32, 14, 5, 88, 6, 12, 1, 54, 7};
7
8      std::size_t idx = 0;
9      while (idx < x.size() && x[idx] != 1)
10     {
11         idx++;
12     }
13
14     if ( idx < x.size() )
15         std::cout << "value 1 found at index " << idx << "\n";
16 }
```

value 1 found at index 12

Conditional repeats

Example - Interactive sum

```
1  #include <iostream>
2
3  int main()
4  {
5      float n, sum = 0.f;
6
7      do
8      {
9          std::cin >> n;
10
11         sum += n;
12     } while (n != 0.f);
13
14     std::cout << sum << "\n";
15 }
```

Conditional flows

Conditional structure - `if ... else`

Purpose:

- Modify the program flow according to an arbitrary condition.
- Flow modification can be associated with an alternative
- Flow selection conditions can be concatenated

General form:

```
1  // Prologue code executed systematically
2
3  if( condition )
4  {
5      // code executed if and only if `condition` evaluates to `true`.
6  }
7
8  // epilogue code systematically executed
```

Conditional structure - if ... else

Purpose:

- Modify the program flow according to an arbitrary condition.
- **The modification of the program flow can be associated with an alternative.**
- Flow selection conditions can be concatenated

General form:

```
1  if( condition )
2  {
3      // code executed if and only if `condition` evaluates to `true`
4  }
5  else
6  {
7      // code executed if and only if `condition` evaluates to `false`.
8  }
```


Conditional structure - if ... else

Purpose:

- modify the program flow according to an arbitrary condition.
- Flow modification can be associated with an alternative
- **The conditions for choosing the flow can be concatenated.**

Forme générale:

```
1  if( condition1 )      { /* code executed if `condition1` evaluates to `true` */ }
2  else if( condition2 ) { /* else code executed if `condition2` evaluates to `true` */ }
3  else if( condition3 ) { /* else code executed if `condition3` evaluates to `true` */ }
4  else
5  {
6      // code executed as a last resort
7  }
```

Conditional structure - if ... else

Example:

```
1  #include <iostream>
2
3  int main()
4  {
5      int n;
6
7      std::cout << "Give an integer: ";
8      std::cin >> n; // read an integer from the keyboard
9
10     if(n % 2 == 0) std::cout << n << "is even.\n";
11     else std::cout << n << " is odd.\n";
12 }
```

Conditional structure - if ... else

Example:

```
1  #include <iostream>
2
3  int main()
4  {
5      int n;
6
7      std::cout << "Give an integer: ";
8      std::cin >> n; // read an integer from the keyboard
9
10     if(n == 0) std::cout << n << "is null.\n";
11     else if(n % 2 == 0) std::cout << n << " is even.\n";
12     else std::cout << n << " is odd.\n";
13 }
```

Conditional structure - if ... else

Example - nested if :

```
1  #include <iostream>
2
3  int main()
4  {
5      int n;
6
7      std::cout << "Give an integer: ";
8      std::cin >> n; // read an integer from the keyboard
9
10     if(n % 2 == 0)
11     {
12         if(n == 0) std::cout << n << "is null.\n";
13         else std::cout << n << "is even.\n";
14     }
15     else std::cout << n << "is odd.\n";
16 }
```

Selective structure - `if ... else`

Internal initialization

- A variable can be initialized wherever the condition is expected.
- The variable will therefore only be accessible in the various `if` alternatives.
- Without additional code, the implicit condition is that the variable is other than 0.

```
1  // i does not yet exist here
2  if( int i = roll(0,10) )
3  {
4      std::cout << i << "is non-null"; // i is accessible
5  }
6  else
7  {
8      std::cout << i << " is null"; // i is accessible
9  }
10
11 // i no longer exists there
```

Selective structure - `if ... else`

Internal initialization

- A variable can be initialized wherever the condition is expected.
- The variable will therefore only be accessible in the various `if` alternatives.
- Without additional code, the implicit condition is that the variable is other than 0.

```
1  // i doesn't exist here yet
2  if( int i = roll(0,10); i > 5 )
3  {
4      std::cout << "Won! " << i " > 5\n"; // i is accessible
5  }
6  else
7  {
8      std::cout << "Lost :( " << i " <= 5\n"; // i is accessible
9  }
10
11 // i no longer exists there
```

Selective structure - switch

Purpose:

- Modify the control flow by selecting a case from N
- Cases are described as an enumeration of possibilities.
- Choice type must be an integer or an enumeration

Forme générale :

```
1  switch( value )
2  {
3      case case1 : // case 1 code
4          break;
5
6      case cas2 : // code for case 2
7          break;
8
9      default : // default code
10 }
```

Structure sélective - switch

Exemple :

```
1  int day;
2  std::cin >> day; // reads the day's number from the keyboard
3
4  switch (day)
5  {
6      case 1: std::cout << "Monday \n"; break;
7      case 2: std::cout << "Tuesday\n"; break;
8      case 3: std::cout << "Wednesday\n"; break;
9      case 4: std::cout << "Thursday"; break;
10     case 5: std::cout << "Friday\n"; break;
11     case 6: std::cout << "Saturday\n"; break;
12     case 7: std::cout << "Sunday\n"; break;
13     default: std::cout << "impossible day";
14 }
```


Selective structure - switch

Example - Semantics of *fallthrough*:

```
1  int day;
2  std::cin >> day; // reads the day's number from the keyboard
3
4  // Tuesday: red tie
5  // Friday: grey tie
6  // weekends: blue tie
7  // otherwise: no tie
8  switch (day)
9  {
10     case 2: std::cout << "red tie\n"; break;
11     case 5: std::cout << "grey tie\n"; break;
12     case 6:
13
14     case 7: std::cout << "blue tie\n"; break;
15     default: std::cout << "no tie\n";
16 }
```

Selective structure - switch

Example - Semantics of *fallthrough*:

```
1  int day;
2  std::cin >> day; // reads the day's number from the keyboard
3
4  // Tuesday: red tie
5  // Friday: grey tie
6  // weekends: blue tie
7  // otherwise: no tie
8  switch (day)
9  {
10     case 2: std::cout << "red tie\n"; break;
11     case 5: std::cout << "grey tie\n"; break;
12     case 6:
13         [[fallthrough]];
14     case 7: std::cout << "blue tie\n"; break;
15     default: std::cout << "no tie\n";
16 }
```

Selective structure - switch

Example - Interaction with enum:

```
1  enum class day { Monday = 1, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday};
2
3  day j = current_day();
4
5  switch (day)
6  {
7      case day::Tuesday:    std::cout << "red tie\n"; break;
8      case day::Friday:    std::cout << "blue tie\n"; break;
9
10     case day::Saturday:
11         [[fallthrough]];
12     case day::Sunday:    std::cout << "gray tie\n"; break;
13
14     default: std::cout << "no tie\n";
15 }
```

Selective structure - switch

Example - Internal initialization:

```
1  enum class day { Monday = 1, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday};
2
3  switch (day j = current_day())
4  {
5      case day::Tuesday:    std::cout << "red tie\n"; break;
6      case day::Friday:    std::cout << "blue tie\n"; break;
7
8      case day::Saturday:
9          [[fallthrough]];
10     case day::Sunday:    std::cout << "gray tie\n"; break;
11
12     default: std::cout << "no tie\n";
13 }
14
15 // j doesnt exist
```

Structured types

Structured types - Enumerations

Principles

- An enumeration abstracts a discrete set of numerical values.
- These values share the same semantics

Use cases

- Replacing “magic” values
- Create integer types with strong semantics

General form

```
enum class identifier { value1, ..., valueN };
```

Structured types - Enumerations

Replacing magic values

```
1  // Classic code with magic value
2  int main()
3  {
4      int day = 4;
5
6      switch(day)
7      {
8          case 1 : std::cout << "it's Monday !\n"; break;
9          case 2 : std::cout << "it's Tuesday !\n"; break;
10         case 3 : std::cout << "it's Wednesday !\n"; break;
11         case 4 : std::cout << "it's Thursday !\n"; break;
12         case 4 : std::cout << "it's Friday !\n"; break;
13         default: std::cout << "it's the weekend!"; break;
14     }
15 }
```

Structured types - Enumerations

Replacing magic values

```
1  // Enumeration-based code
2  enum class day { monday = 1 , tuesday, wednesday, thursday, friday, saturday, sunday};
3
4  int main()
5  {
6      day j = day::thursday;
7
8      switch(j)
9      {
10         case day::monday : std::cout << "it's Monday !\n"; break;
11         case jour::tuesday : std::cout << "it's Tuesday"; break;
12         case day::wednesday : std::cout << "it's wednesday"; break;
13         case jour::thursday : std::cout << "it's thursday"; break;
14         case jour::Friday std::cout << "it's friday"; break;
15         default: std::cout << "it's the weekend!"; break;
16     }
17 }
```


Structured types - Enumerations

Replacing magic values

```
1  // Enumeration-based code
2  enum class day { monday = 1 , tuesday, wednesday, thursday, friday, saturday, sunday};
3
4  int main()
5  {
6      day j = 5; // does not compile because day is not an int
7
8      switch(j)
9      {
10         case day::monday : std::cout << "it's Monday !\n"; break;
11         case jour::tuesday : std::cout << "it's Tuesday"; break;
12         case day::wednesday : std::cout << "it's wednesday"; break;
13         case jour::thursday : std::cout << "it's thursday"; break;
14         case jour::Friday std::cout << "it's friday"; break;
15         default: std::cout << "it's the weekend!"; break;
16     }
17 }
```

Structured types - Enumerations

Replacing magic values

- It is possible to specify the integer type used to store the value of enumerations.
- The compiler checks that the values used are representative of the type used.

```
1  // Enumeration-based code
2  enum class jour : std::uint8_t
3  {
4      monday = 1 , tuesday, wednesday, thursday, friday, saturday, sunday
5  };
6
7  day j = day::thursday;
```

Structured types - Enumerations

Replacing magic values

- It is possible to specify the integer type used to store the value of enumerations.
- The compiler checks that the values used are representative of the type used.

```
// Enumeration-based code
enum class jour : std::uint8_t
{
    monday = 1 , tuesday, wednesday, thursday, friday, saturday
    // error: enumerator value '1500' is outside the range of underlying type 'unsigned char'
    sunday = 1500
};

day j = day::thursday;
```

Structured types - Enumerations

Strong semantic integer type

- It is possible to specify the integer type used to store the value of enumerations.
- It's possible to provide no enumerated value
- These two rules allow you to build **strong semantic integer types**.

```
1  // Enumeration-based code
2  enum class age : std::uint8_t {};
3
4  // Age is constructed explicitly from an integer
5  age age_bebe{}; // age_bebe is 0
6  age age_ado{12}; // age_ado is 12
7
8  // Ages and integers don't mix: These codes don't compile:
9  age age_papy = 90; // error: cannot convert 'int' to 'age' in initialization
10 int n = age_ado / 2; // error: no match for 'operator/' (operand types are 'age' and 'int')
```

Structured types - Aggregates

Principles

- Defines an entity with precise semantics
- The elements of an aggregate are its **data members**.
- Allows new types to be added to the language

Essential elements

- Defining an aggregate
- Know the default behaviors

Structured types - Aggregates

General form

```
1  struct type_name
2  {
3      type_member1 member_1;
4      type_member2 member_2;
5
6      // etc ....
7
8      type_memberN member_N;
9  };
```

- The number and type of members is arbitrary.
- The name of the aggregate or its members is arbitrary.
- An aggregate can contain one or more aggregates.

Structured types - Aggregates

General form

```
1  // Simple aggregate
2  struct point3D
3  {
4      double x; // x is a double, member of point3D
5      double y;
6      double z;
7  };
8
9  // Composite aggregate
10 struct weighted_point3D
11 {
12     point3D point; // Note the reuse of point3D
13     int weight;    // weight is an additional member
14 };
```

Structured types - Aggregates

Construction

```
1  #include <iostream>
2
3  struct point3D
4  {
5      double x,y, z;
6  };
7
8  int main()
9  {
10     // Initialization of each member in order of definition
11     point3D p = { 1.5, -0.654, 2.7 };
12
13     std::cout << p.x << " " << p.y << " " << p.z << "\n";
14 }
```

1.5 -0.654 2.7

Structured types - Aggregates

Construction

```
1  #include <iostream>
2
3  struct point3D
4  {
5      double x,y, z;
6  };
7
8  int main()
9  {
10     // You can 'forget' members
11     point3D p = { 9.99 };
12
13     std::cout << p.x << " " << p.y << " " << p.z << "\n";
14 }
```

9.99 0 0

Structured types - Aggregates

Construction

```
1  #include <iostream>
2
3  struct point3D
4  {
5      double x,y, z;
6  };
7
8  int main()
9  {
10     // You can 'forget' all the members
11     point3D p = {};
12
13     std::cout << p.x << " " << p.y << " " << p.z << "\n";
14 }
```

0 0 0

Structured types - Aggregates

Construction

```
1  #include <iostream>
2
3  struct point3D
4  {
5      double x,y, z;
6  };
7
8  int main()
9  {
10     // Be careful not to use an uninitialized variable!
11     point3D p;
12
13     std::cout << p.x << " " << p.y << " " << p.z << "\n";
14 }
```

2.07455e-317 2.07444e-317 6.95283e-310

Structured types - Aggregates

Construction

```
1  #include <iostream>
2
3  // Review of point3D and weighted_point3D definitions
4
5  int main()
6  {
7      // Internal aggregates are initialized using {}.
8      weighted_point3D p{{1,1,1},9};
9
10     std::cout << p.point.x << " " << p.point.y << " " << p.point.z << "\n";
11     std::cout << p.poids << "\n";
12 }
```

```
1 1 1
9
```

Structured types - Aggregates

Default values

```
1  #include <iostream>
2
3  struct point3D
4  {
5      double x = 1.;
6      double y = 2.;
7      double z = 3.;
8  };
9
10 int main()
11 {
12     point3D p;
13     std::cout << p.x << " " << p.y << " " << p.z << "\n";
14 }
```

1 2 3

Imperative aspects

Notion of functions

- Parameter management
- Multiple returns
- Interaction with structures

Functional polymorphism

- Function and operator overloading
- Generic functions

Notion of functions

Notion of functions

Basic principles

- A function encapsulates know-how
- It's a group of reusable expressions
- A function consumes **parameters** to produce a **return value**. **return value** * Using a function
- Using a function requires a **function call**.

General shape

```
1  return_type function_name( type_1 parameter_1, ..., type_N parameter_N )
2  {
3      expression;
4      // ...
5      expression;
6
7      return return_value;
8  }
```

Basic principles

Example

```
1  #include <iostream>
2
3  // squared_difference consumes two int and returns a new int
4  int squared_difference(int a, int b)
5  {
6      auto diff = a - b;    // Expression
7      return diff * diff;  // Return function
8  }
9
10 int main()
11 {
12     auto x = squared_difference(6, 10); // Call function with parameters 6 and 10
13     std::cout << x << "\n";           // x contains the value returned by the
14 }
```

16

Basic principles

Example

```
1  #include <iostream>
2
3  // void indicates that the function returns nothing
4  void tell_me(float x)
5  {
6      std::cout << "Value of x = " << x << "\n";
7      // return pass
8  }
9
10 int main()
11 {
12     auto u = 3.14159f;
13     tell_me(u);
14 }
```

Valeur de x = 3.14159

Basic principles

Recursive function

- A function can call itself
- It is necessary to manually manage its terminal case(s)

Example

```
1  int fibonacci(int n)
2  {
3      if(n<2) return 1; else return fibonacci(n-1) + fibonacci(n-2);
4  }
5
6  int main()
7  {
8      std::cout << fibonacci(10) << "\n";
9  }
```

Changing parameters

Passing by value

- The function receives a **copy** of its parameters
- Modifications are not reflected in the parameters themselves

```
1 void f(double a)
2 {
3     a /= 2.5f;
4     std::cout << a << "\n";    // Displays 10.3
5 }
6
7 int main()
8 {
9     double x = 25.75;
10    std::cout << x << "\n";    // Displays 25.75
11    f(x);
12    std::cout << x << "\n";    // Displays 25.75
13 }
```

Changing parameters

Passing by reference

- The function receives a **reference** to its parameters
- Modifications are reflected in the parameters themselves

```
1 void f(double& a)
2 {
3     a /= 2.5f;
4     std::cout << a << "\n";    // Displays 10.3
5 }
6
7 int main()
8 {
9     double x = 25.75;
10    std::cout << x << "\n";    // Displays 25.75
11    f(x);
12    std::cout << x << "\n";    // Displays 10.3
13 }
```

Changing parameters

Passing by immutable reference

- function receives an **immutable reference** to its parameters
- Parameters cannot be modified

```
1 void f(double const& a)
2 {
3     // a /= 2.5f; --> compilation error
4     std::cout << a << "\n";    // Affiche 25.75
5 }
6
7 int main()
8 {
9     double x = 25.75;
10    std::cout << x << "\n";    // Displays 25.75
11    f(x);
12    std::cout << x << "\n";    // Displays 25.75
13 }
```

Changing parameters

Special case of pointer parameter

- Indicates that the parameter can be omitted
- Behavior similar to a reference

```
1 void f(double* a)
2 {
3     if(a != nullptr)
4         *a /= 2.5f;
5 }
6
7 int main()
8 {
9     double x = 25.75;
10    std::cout << x << "\n";    // Displays 25.75
11    f(&x);                     // Passing address
12    std::cout << x << "\n";    // Displays 10.3
13 }
```


Default settings

Principle

- It is possible to specify a default value for one or more parameters.
- These parameters must be grouped at the end of the parameter list.

```
1  #include <iostream>
2
3  void display(bool cursor, char c = '*', int n = 1)
4  {
5      if(cursor)
6          std::cout << "> ";
7
8      for (int i = 1; i <= n; ++i)
9          std::cout << c;
10
11     std::cout << "\n";
12 }
```

Default settings

Principle

- It is possible to specify a default value for one or more parameters.
- These parameters must be grouped at the end of the parameter list.

```
1  int main()  
2  {  
3      display(true, '+', 3);  
4      display(false, '$', 5);  
5      display(false, '!');  
6      display(true);  
7  }
```

```
>+++  
$$$$$  
!  
> *
```

Interaction with structures

Passing an aggregate as a function parameter

- Aggregates are passed as parameters in a natural way
- Special case of pointer parameter

```
1  struct fraction
2  {
3      int num, denum;
4  };
5
6  float value(fraction a)
7  {
8      auto div = static_cast<float>(a.denum); // Converting a.denum to float
9      return a.num / div;
10 }
```

Interaction with structures

Passing an aggregate as a function parameter

- Aggregates are passed as parameters in a natural way
- **Special case of pointer parameter**

```
1  struct fraction
2  {
3      int num, denum;
4  };
5
6  float value_or(fraction* a, float v)
7  {
8      if( a != nullptr )           // a available?
9          return a->num / static_cast<float>(a->denum); // -> replaces .
10     else                          // otherwise return the 2nd parameter
11         return v;
12 }
```

Interaction with structures

Passing an aggregate as a function parameter

- Aggregates are passed as parameters in a natural way
- **Special case of pointer parameter**

```
1  #include <iostream>
2
3  int main()
4  {
5      fraction r{1,4};
6      std::cout << value_or(&r, 0.f) << "\n";
7      std::cout << value_or(nullptr, 13.37f) << "\n";
8  }
```

```
0.25
13.37
```

Interaction with structures

Return a structure from a function

- Explicit or implicit
- Direct or piecewise

```
1  // Return by implicit construction
2  fraction identity()
3  {
4      return {0,1};
5  }
6
7  // Return by explicit construction
8  fraction identity()
9  {
10     return fraction{0,1};
11 }
```

Interaction with structures

Return a structure from a function

```
1  // Return by implicit direct construction
2  fraction multiply(fraction a, fraction b)
3  {
4      return {a.num*b.num, a.denum*b.denum};
5  }
6
7  // Return by piece-by-piece construction
8  fraction add(fraction a, fraction b)
9  {
10     fraction that;
11
12     that.num    = a.num*b.denum + a.denum*b.num;
13     that.denum  = a.denum * b.denum;
14
15     return that;
16 }
```

Interaction with structures

Notion of member function

- A function can have semantics that make it indissociable from the type it manipulates.
- For clarity's sake, the function's **code** must be associated with the aggregate's **code**.
- In this case, we define a **member function**.

```
1  fraction x{178, 16};  
2  
3  x.simplify();  // x is simplified "in place"
```

Problems

- Syntactic definition of a member function
- Property of member functions with respect to the aggregate

Interaction with structures

Syntax of member functions

```
1  struct fraction
2  {
3      int num, denum;
4
5      // Declaration
6      void scale(int factor);
7  };
8
9  // Definition - Note the fraction prefix:: which associates scale with the fraction type
10 void fraction::scale(int factor)
11 {
12     // the num used here is that of the aggregate calling this member function
13     num *= factor;
14 }
```

Interaction with structures

Syntax of member functions

```
1  struct fraction
2  {
3      int num, denum;
4
5      // On-line definition
6      // If the function code is trivial, it's acceptable to define
7      // the function directly within the aggregate
8      void scale(int factor)
9      {
10         num *= factor;
11     }
12 };
```

Interaction with structures

Syntax of member functions

```
1  int main()
2  {
3      fraction x{17,23};
4
5      std::cout << x.num << " / " << x.denum << "\n";
6
7      x.scale(10);
8      std::cout << x.num << " / " << x.denum << "\n";
9  }
```

```
17 / 23
170 / 23
```

Interaction with structures

Notion of immutable function

- How to respect the immutability of an aggregate instance?

```
1  struct fraction
2  {
3      int num, denum;
4
5      double value() { return num / static_cast<double>(denum); }
6  };
7
8  fraction x{4,7};
9  auto v = x.value(); // OK
```

Interaction with structures

Notion of immutable function

- How to respect the immutability of an aggregate instance?

```
1  struct fraction
2  {
3      int num, denum;
4
5      double value() { return num / static_cast<double>(denum); }
6  };
7
8  fraction const x{4,7};
9  // Compilation error:
10 // error: 'this' argument to member function 'value' has type 'const fraction'
11 // but function is not marked const
12 //
13 auto v = x.value();
```

Interaction with structures

Notion of immutable function

- How to respect the immutability of an aggregate instance?

```
1  struct fraction
2  {
3      int num, denum;
4
5      // value() does not modify the fraction state
6      // It is therefore compatible with the immutable nature of a fraction instance.
7      // This must be indicated by the const keyword
8      //
9      //
10     //
11     double value() const { return num / static_cast<double>(denum); }
12 };
13
14 fraction const x{4,7};
15 auto v = x.value();    // OK
```

Multiple return

Problem

- Some functions need to return multiple results.
- How to specify this return?
- How to retrieve these values?

```
/* ??? */ quotient_reste( int a, int b )  
{  
    return /* ??? */;  
}
```

Solutions

- Return an aggregate
- Use of `std::tie`
- Structural decomposition

Multiple return

Aggregate return

- Simple to use
- Requires one aggregate type per function

```
1  struct sortie_qr
2  {
3      int quotient, reste;
4  };
5
6  sortie_qr quotient_reste( int a, int b )
7  {
8      return {a/b, a%b};
9  }
10
11  auto x = quotient_reste(17,4);
12  std::cout << x.quotient << " " << x.reste << "\n";
```


Multiple return

Using `std::tie`

- Simple to use
- Sub-sets have no clear semantics

```
1  #include <tuple>
2
3  std::tuple<int,int> quotient_reste( int a, int b )
4  {
5      return {a/b, a%b};
6  }
7
8  int q,r;
9  std::tie(q,r) = quotient_reste(17,4);
10 std::cout << q << " " << r << "\n";
```

Multiple return

Structural decomposition

- Syntax: `auto [name1,name2,...,nameN] = f();`
- `f` must return a value similar to a tuple or aggregate
- This syntax **defines** variables instead of reusing them

```
1  #include <tuple>
2
3  std::tuple<int,int> quotient_reste( int a, int b )
4  {
5      return {a/b, a%b};
6  }
7
8  auto [q,r] = quotient_reste(17,4);
9  std::cout << q << " " << r << "\n";
```

Functional polymorphism

Function overload

Motivations

- Why should functions performing a similar operation on different types have different names?
- Simplify code use by making it intuitive
- First steps towards a notion of genericity

Notion of *ad-hoc* polymorphism

- Ability to associate a **function name** with different behaviors
- Setting up a type-based resolution system
- The ability to combine functions with identical names in the same program

Function overload

Principles

- A C++ function is uniquely identifiable by its **signature**.
- Signature = Function name + List of parameter types
- Several functions with the same **name** but **different** signatures can therefore coexist.

The return type plays no role in the signature of a function

Exemple

```
1 double f();           // signature: f + { }
2 double f(int a);      // signature: f + { int }
3 char f(char x, char y); // signature: f + { char, char }
4
5 int f();              // signature: f + { } -- collision with double f()
```

Function overload

Installation

```
1  std::string as_text(std::string const& s) { return "'" + s + "'";    }
2  std::string as_text(char  c)              { return std::string(1, c); }
3  std::string as_text(float f)
4  {
5      std::ostringstream o;
6      o << f << "f";
7      return o.str();
8  }
9
10 std::cout << as_text('Z')           << "\n";
11 std::cout << as_text(3.5f)          << "\n";
12 std::cout << as_text("un test de texte") << "\n";
```

```
Z
3.5f
'un test de texte'
```

Operator overload

Motivation

- Give user-defined types a natural interface
- Make user-defined types transparent to the language

Principles

- An operator is a function with an infix syntax.

$$a = b + c \Rightarrow \text{operator}=(a, \text{operator}+(b,c))$$

- Operators can be overloaded as functions as long as at least one at least one parameter is a user type

Operator overload - Overloadable operators

Operations	Operators	General form
Logical unary	!	bool operator!(T a)
Unary arithmetic	-,~	R operator<@>(T a)
Binary arithmetic	+, -, *, /, %	R operator<@>(T a, U b)
Binary bitwise	<<, >>, ^, &,	R operator<@>(T a, U b)
Comparison	==, !=, <, >, <=, >=	bool operator<@>(T a, U b)
Logic	&&,	bool operator<@>(T a, U b)

An overloaded operator can behave in any way.
It's good practice to keep semantics close to the original.

Operator overload

Example

```
1  struct fraction
2  {
3      int num, denum;
4  };
5
6  fraction operator-(fraction const& arg)
7  {
8      return { -arg.num, arg.denum };
9  }
10
11 fraction operator+(fraction const& lhs, fraction const& rhs)
12 {
13     auto num    = lhs.num*rhs.denum + lhs.denum*rhs.num;
14     auto denum  = lhs.denum*rhs.denum;
15
16     return {num, denum};
17 }
```

Operator overload

Example

```
1  fraction operator+(fraction const& lhs, int rhs)
2  {
3      auto num    = lhs.num + lhs.denum*rhs;
4      auto denum  = lhs.denum;
5
6      return {num, denum};
7  }
8
9  fraction operator+(int lhs, fraction const& rhs)
10 {
11     auto num    = lhs*rhs.denum + lhs*rhs.num;
12     auto denum  = rhs.denum;
13
14     return {num, denum};
15 }
```

Operator overload - Interaction with flows

Example

```
1  #include <iostream>
2
3  std::ostream& operator<<(std::ostream& os, fraction const& f)
4  {
5      os << f.num << "/" << f.denum;
6      return os;
7  }
8
9  std::istream& operator>>(std::istream& is, fraction& f)
10 {
11     is >> f.num >> f.denum;
12     return is;
13 }
```

Generic functions

Motivation

- Large amount of overhead for a given function
- Extremely repetitive code

```
1  double minimum(double a, double b) { return a<b ? a : b; }
2  float  minimum(float a, float  b) { return a<b ? a : b; }
3  int    minimum(int   a, int    b) { return a<b ? a : b; }
4  char   minimum(char  a, char   b) { return a<b ? a : b; }
5  short  minimum(short a, short  b) { return a<b ? a : b; }
```

This type of code does not scale and is potentially
a complex source of error to trace *a posteriori*.

Generic functions

Objectives

- Provide a syntax for expressing the universality of an overloaded function
- Limit the impact on code written, maintained and generated

Solutions

- Notion of **function patterns**.
- Syntax known as function *templates*.

```
1  template<typename Type>
2  Type  minimum(Type  a, Type  b)
3  {
4      return a<b ? a : b;
5  }
```

Generic functions

Principles

- Replacement of types by aparametric types
- The function call forces the compiler to deduce the actual type of these parameters by analyzing the type of the values passed as parameters.
- The exact code is regenerated from this deduction.
- If this deduction is impossible, a compilation error occurs.

```
1  template<typename Type>
2  Type minimum(Type a, Type b)
3  {
4      return a < b ? a : b;
5  }
6
7  // Type is deduced as `int`
8  auto x = minimum(4, 5);
9
10 // Type is deduced as `float`
11 auto y = minimum(4.6f, -0.5f);
12
13 // Incorrect case:
14 // error: no matching function
15 // for call to 'minimum(double, int)'
16
17 auto z = minimum(4.6, 5);
```

Generic functions

template parameter

- Introduced by the notation `template< >`.
- Contains at least one parameter of type
- These parameters have a unique identifier by the keyword `typename`.

```
1  //      1st template param.
2  //      |      2nd template param.
3  //      |      |      3rd template param.
4  //      |      |      |
5  //      v      v      v
6  template<typename Src, typename Dst, typename Size>
7  void copy(Src const& src, Dst& dst, Size qty)
8  {
9      // Use Size as index type i
10     for(Size i=0;i<qty;++i)
11         dst[i] = src[i];
12 }
```

Return type inference

Principles

- The return type of generic functions can be complex to express or even impossible for the developer to determine
- The compiler has all the information it needs to determine the exact return type.
- A syntax is introduced to leave this task to the compiler.

Syntax elements

- Keyword **auto**
- Keyword **decltype**

Operating mode

- Explicit: delayed calculation of return type
- Implicit: computation entirely delegated to compiler

Return type inference

Example:

```
1  template<typename T1, typename T2>
2  /* QUID ??? */ addition(T1 a, T2 b)
3  {
4      return a + b;
5  }
```

Explicite case:

```
1  template<typename T1, typename T2>
2  // auto indicates that the type will be calculated later
3  // |
4  // v
5  auto addition(T1 a, T2 b)
6      -> decltype(a+b) // <--- decltype evaluates the type of its parameter
7  {
8      return a + b;
9  }
```

Return type inference

Example:

```
1  template<typename T1, typename T2>
2  /* QUID ??? */ addition(T1 a, T2 b)
3  {
4      return a + b;
5  }
```

Implicit case:

```
1  template<typename T1, typename T2>
2  // auto indicates that the type will be calculated by the compiler
3  // |
4  // v
5  auto addition(T1 a, T2 b)
6  {
7      // The compiler evaluates the type of expression returned by return
8      return a + b;
9  }
```

The Standard Library

Algorithms

Objectives

- Make data sequence manipulation natural
- Provide non-trivial algorithms
- Increase code expressiveness and maintainability
- Increase code cohesion

Solutions

- Standard algorithms
- Anonymous functions

Setting the scene - Initial code

```
1  bool match_pattern(MemoryBuffer const& m)
2  {
3      return m.size() > 2 && m[0] == 'E' && m[1] == 'Z';
4  }
5
6  bool process_buffer(std::vector<MemoryBuffer> const& mems)
7  {
8      for(std::size_t i = 0; i< mems.size(); ++i)
9      {
10         if( match_pattern(mems[i]) )
11             return true;
12     }
13
14     return false;
15 }
```

Case study - Interval loop

- Abstraction of the iteration process
- Generalized code for any container

```
1  bool match_pattern(MemoryBuffer const& m)
2  {
3      return m.size() > 2 && m[0] == 'E' && m[1] == 'Z';
4  }
5
6  bool process_buffer(std::vector<MemoryBuffer> const& mems)
7  {
8      for(auto const& m : mems)
9      {
10         if( match_pattern(m) ) return true;
11     }
12
13     return false;
14 }
```

Algorithms as loop abstraction

Objectives

- Provide a reference implementation of classical processing on sequences
- Complete abstraction of sequence type
- Maximize the genericity of code manipulating a sequence

Practical application

- Notion of **iterator**
- Optimized compilation implementation
- Wide range of algorithms

Case study - Iterators

```
1  bool match_pattern(MemoryBuffer const& m)
2  {
3      return m.size() > 2 && m[0] == 'E' && m[1] == 'Z';
4  }
5
6  bool process_buffer(std::vector<MemoryBuffer> const& mems)
7  {
8      std::vector<MemoryBuffer>::const_iterator b = mems.cbegin();
9      std::vector<MemoryBuffer>::const_iterator e = mems.cend();
10
11     while(b != e)
12     {
13         if( match_pattern(*b++) ) return true;
14     }
15
16     return false;
17 }
```


Standard algorithms

Predicates

- `all_of`, `any_of`, `none_of`
- `count`, `count_if`

Modification

- `transform`, `for_each`
- `copy`, `replace`, `remove`
- `copy_if`, `replace_if`, `remove_if`

Search

- `find`, `find_if`
- `search`, `search_if`
- `max_element`, `min_element`, `minmax_element`

Standard algorithms

Numerical calculations

- `accumulate`, `reduce`, `transform_reduce`
- `inclusive_scan`, `exclusive_scan`
- `inner_product`

Structural modification

- `reverse`, `rotate`, `shuffle`, `sample`
- `unique`, `fill`

Sorting

- `sort`, `stable_sort`, `partial_sort`, `nth_element`
- `partition`, `stable_partition`
- `is_sorted`, `is_partitioned`

Case study - Algorithm

```
1  bool match_pattern(MemoryBuffer const& m)
2  {
3      return m.size() > 2 && m[0] == 'E' && m[1] == 'Z';
4  }
5
6  bool process_buffer(std::vector<MemoryBuffer> const& mems)
7  {
8      auto it = std::find_if(mems.begin(), mems.end(), match_pattern);
9      return it != mems.end();
10 }
11
12 bool process_buffer2(std::vector<MemoryBuffer> const& mems)
13 {
14     return std::any_of(mems.begin(), mems.end(), match_pattern);
15 }
```

Anonymous functions

Algorithm interface limitations

- Passing a function as a parameter is limited
- Impossible to use an overloaded function
- Cannot use a generic function

Solutions

- Function objects
- Anonymous functions

Anonymous functions - Syntax

Definition of an anonymous function

- An anonymous function is the function equivalent of a temporary variable.
- It defines a fragment of code that can be used “on the spot”.

```
1  [ ](int a, float b, auto c)
2  {
3      auto x = (a+b)/2;
4      return x<c ? c : x;
5  }
```

Anonymous functions - Syntax

Definition of an anonymous function

- An anonymous function contains arbitrary code just like a normal function.
- The only limitation is that it cannot be called recursively

```
1  [ ](int a, float b, auto c)
2  {
3      // Function body containing arbitrary code
4      auto x = (a+b)/2;
5      return x<c ? c : x;
6  }
```

Anonymous functions - Syntax

Anonymous function definition - Capture

- The definition of an anonymous function begins with its **capture environment*.
- This environment lists variables used inside the function but defined outside.
- By default, this environment is empty.

```
1 // --- Capture environment
2 // |
3 // |
4 // v
5 [ ](int a, float b, auto c)
6 {
7     auto x = (a+b)/2;
8     return x<c ? c : x;
9 }
```

Anonymous functions - Syntax

Anonymous function definition - Capture

- Variables are listed in the environment by name.
- You can capture a variable by reference using &.
- [=] means to capture everything by value
- [&] means capture all by reference

```
1  int u = 9;
2  int k = 0;
3
4  [u,&k](int a, float b, auto c)
5  {
6      auto x = (a+b)/2;
7      k++;           // The exterior k is modified
8
9      return x<c ? u : x; // This u is a copy of the external u
10 }
```


Anonymous functions - Syntax

Anonymous function definition - Parameters

- An anonymous function can have an arbitrary number of parameters.
- They define how the anonymous function will be called

```
1  //      Parameter List
2  //      |
3  //      v
4  //  <----->
5  [ ](int a, float b, auto c)
6  {
7      auto x = (a+b)/2;
8      return x<c ? c : x;
9  }
```

Anonymous functions - Syntax

Anonymous function definition - Parameters

- Parameters can have a concrete type
- Parameters can be generic. In this case, **auto** is used.

```
1  //  Concrete parameters
2  //           |
3  //   +-----+
4  //   v       v
5  [ ](int a, float b, auto c)
6  {
7      auto x = (a+b)/2;
8      return x<c ? c : x;
9  }
```

Anonymous functions - Syntax

Definition of an anonymous function

- Parameters can have a concrete type
- Parameters can be generic. In this case, **auto** is used.

```
1  //          Generic parameter
2  //          |
3  //          |
4  //          v
5  [ ](int a, float b, auto c)
6  {
7      auto x = (a+b)/2;
8      return x<c ? c : x;
9  }
```

Anonymous functions - How to use

Interaction with classic functions

- An anonymous function has no human-readable type.
- Taking an anonymous function as a parameter requires a template parameter

```
1  template<typename Func>
2  void display_twice(Func f, int x)
3  {
4      std::cout << f(x) << "\n" << f(x+1) << "\n";
5  }
6
7  int main()
8  {
9      display_twice( [](auto i) { return 1.f/i; }, 4 );
10 }
```

0.25

0.2

Anonymous functions - How to use

Interaction with classic functions

- An anonymous function has no human-readable type.
- An anonymous function can be returned from a function with an automatic return type.

```
1  template<typename Func> auto square(Func f)
2  {
3      return [f](auto x) { return f(f(x)); };
4  }
5
6  int main()
7  {
8      auto g = square([](auto x) { return x*x; });
9      std::cout << g(5) << "\n";
10 }
```

625

Anonymous functions - Application

Final version

- Code is more localized
- Code is more optimizable

```
1  bool process_buffer2(std::vector<MemoryBuffer> const& mems)
2  {
3      return std::any_of( mems.begin(), mems.end()
4                          , [](MemoryBuffer const& m)
5                          {
6                              return m.size() > 2 && m[0] == 'E' && m[1] == 'Z';
7                          }
8                          );
9  }
```